AULA 02

# Como os agentes trabalham juntos

4 horas de aula
Teoria e prática

# Modelagem e comunicação entre agentes

Módulo 4
Sistemas Multi-Agente

Adriano Andrade

# O estado já tem uma história




Receber, reservar e reconhecer a mesma intenção.

Dividir P001 entre papéis e capacidades.

Definir quem confirma cada mudança.

Retomada de 15 minutos. JSON e deduplicação continuam como base.

# Estado compartilhado e autoridade

## Antes

Uma aplicação controla o fluxo.

O formato da mensagem já é conhecido.

## Agora

Vários papéis propõem mudanças.

Cada transição tem um responsável.

RunnableLambda chama o consumidor visível.

A novidade é decidir quem pode alterar cada estado; JSON e deduplicação já vêm do M3.

# Um vocabulário comum

| Campo | Significado | Exemplo |
|---|---|---|
| item | Produto do catálogo | camiseta |
| quantidade | Unidades do produto | 2 |
| capacidade | Aptidão para uma tarefa | estoque |
| versao | Revisão do plano | 2 |

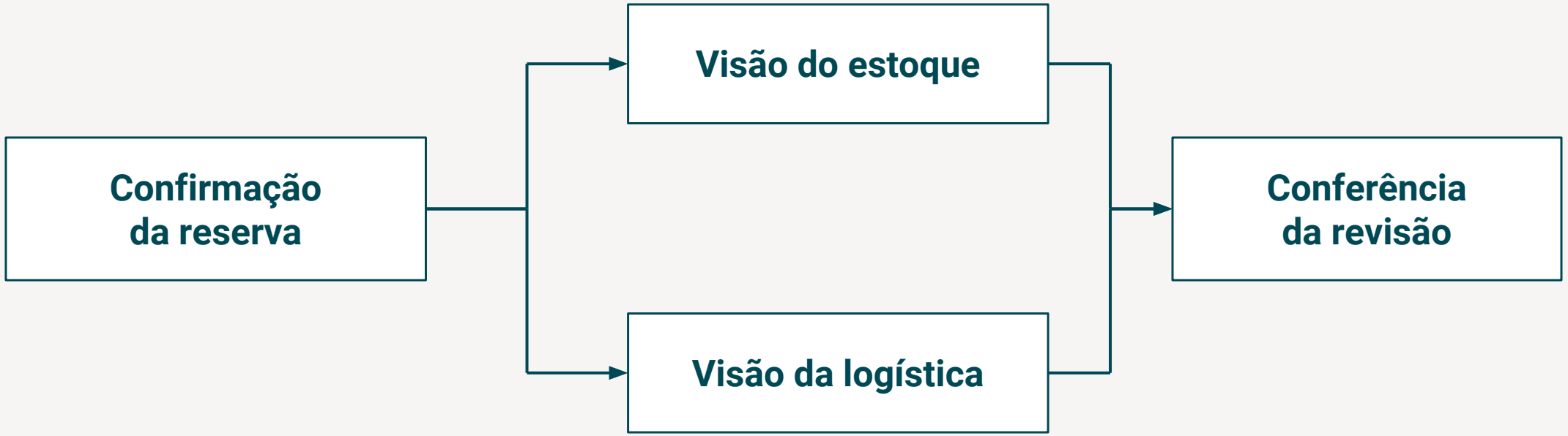
A forma correta do JSON precisa ser acompanhada de significado e unidade.

# O significado precisa acompanhar o número

| Dado | Leitura correta | Confusão a evitar |
| --- | --- | --- |
| quantidade = 2 | Duas camisetas | Duas pessoas na equipe |
| preco = 35 | Reais por camiseta | Duração da tarefa |
| estado = reservado | Etapa do pedido | Pedido já enviado |

Mesmo tipo numérico não significa a mesma grandeza. A mensagem precisa de vocabulário comum.

# O mesmo pedido em visões diferentes




Uma cópia pode estar antiga. O sistema precisa identificar a origem e a versão de cada informação.

# Envelope e conteúdo da mensagem

## Envelope mínimo do núcleo

```
mensagem = {
    "id": "m1",
    "remetente": "Ana",
    "destinatario": "coordenador",
    "acao": "reservar"
}
```

## Contexto da rodada

Pedido P001
Estado inicial: recebido

Quem pode reservar?

O pedido P001 é a entrada comum desta rodada. O envelope didático não autentica o remetente nem implementa FIPA ACL.

# Cada identificador responde a uma pergunta

| Identificador | Pergunta | Exemplo |
|---|---|---|
| pedido_id | Qual solicitação? | P001 |
| id | Qual mensagem? | M02 |
| conversa | Qual rodada? | C03 |
| correlacao | Resposta a qual mensagem? | M01 |

Reenviar a mesma intenção e emitir uma nova mensagem não são a mesma operação.

# Quem tem autoridade nesta simulação?

| Ação | Remetente permitido | Condição do estado |
| --- | --- | --- |
| reservar | Ana | recebido |
| preparar | Bruno | reservado |

Ana representa estoque e Bruno representa logística. A aplicação define essa política antes de receber as mensagens.

## Previsão da primeira mensagem

# Ana pede reservar. O que deve mudar?

P001 está recebido; m1 ainda não apareceu; o destinatário é o coordenador.

Anote o estado seguinte e o motivo. A resposta aparece no rastro depois da execução.

# Pedido, aceite e execução




Solicita reservar duas camisetas.

Aceita ou recusa sob condições.

Registra a reserva que realmente ocorreu.

O aceite estabelece compromisso. A confirmação do efeito exige outra evidência.

# Evento de domínio e envelope padronizado

## Ocorrência da loja

O estoque confirmou uma reserva.

P001 e a quantidade identificam o objeto.

## CloudEvents 1.0.2

id e source identificam a ocorrência.

type e specversion têm significados definidos.

data transporta o conteúdo do domínio.

Um dicionário parecido não basta para declarar implementação do padrão. [S06]

# Duplicata e conflito pedem respostas diferentes

| Recebimento | O que conferir | Decisão possível |
| --- | --- | --- |
| Mesmo id | Resultado já conhecido | Reutilizar a resposta |
| Novo id, mesma intenção | Efeito e transição existentes | Evitar novo efeito |
| Duas confirmações incompatíveis | Autoridade e versão | Recusar ou aguardar |

A deduplicação local não escolhe quem tem autoridade sobre o estado.

# Mesmo ID não é a mesma coisa que mesmo pedido

| Comparação | O que identifica | Decisão local |
|---|---|---|
| P001 em m1 e m2 | Uma solicitação | Pode conter ações diferentes |
| m1 chega duas vezes | A mesma mensagem | Não reaplicar a transição |
| Novo ID, mesma ação | Outra mensagem | Conferir o estado atual |

O núcleo mantém IDs vistos e verifica o antecedente. Isso não é garantia geral de entrega exatamente uma vez.

# O consumidor confere antes de alterar

## Recorte do código

```
def consumir(mensagem):
    if mensagem["id"] in vistos:
        return "duplicada"
    regra = permissoes.get(mensagem["acao"])
    if (regra is None or
        mensagem["remetente"] != regra[0]):
        return "sem_autoridade"
    # O consumidor continua no caderno.
```

## O que observar

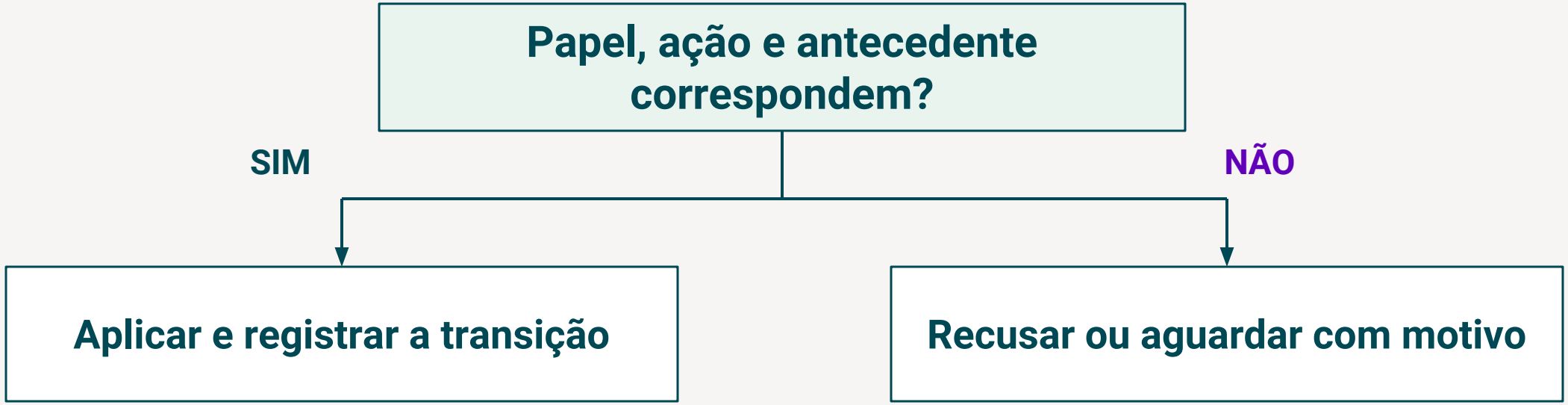
ID já aceito
→ duplicada

Ação ou papel indevido
→ sem_autoridade

Primeiro recorte: ainda faltam destino, estado e aplicação da transição. Não execute esse trecho isolado como consumidor completo.

# Uma confirmação pode alterar o estado?

Papel permitido / Antecedente confirmado / Mensagem para este consumidor




Este núcleo verifica identidade declarada, autoridade local, ordem e duplicata. A versão do plano será tratada na Aula 5.

# A condição do estado também faz parte da regra

## Recorte do código

```
regra = permissoes[mensagem["acao"]]
pode = (mensagem["remetente"] == regra[0]
        and estado["valor"] == regra[1]
        and mensagem["destinatario"] == "coordenador")
if pode:
    estado["valor"] = regra[2]
    vistos.add(mensagem["id"])
```

## O que observar

Papel permitido
+ estado esperado
+ destinatário correto

Então aplicar e marcar o ID.

Recorte da transição com ação reconhecida e ID novo. O consumidor completo recusa os demais casos antes de modificar o dicionário.

# Rastro da fila de P001

| Chegada | Decisão | Estado após a leitura |
|---|---|---|
| Ana: reservar / m1 | aceita | reservado |
| Bruno: reservar / m2 | sem_autoridade | reservado |
| Bruno: preparar / m3 | aceita | preparado |
| Ana: reservar / m1 | duplicada | preparado |

A mesma mensagem não cria outra transição. Um novo ID não dá permissão ao papel errado.

# Causalidade e ordem de chegada




A reserva se torna evidência para a etapa seguinte.

Outras mensagens podem chegar antes dela.

O consumidor confere o antecedente.

O horário não prova causalidade. A relação aconteceu-antes estabelece uma ordem parcial. [S08]

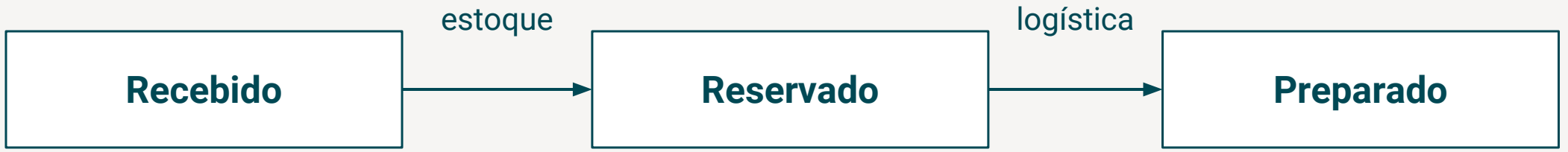
## E se preparar chegar primeiro?

# Bruno pode preparar um pedido ainda recebido?

A identidade está permitida para preparar, mas a confirmação de reserva ainda não foi recebida.

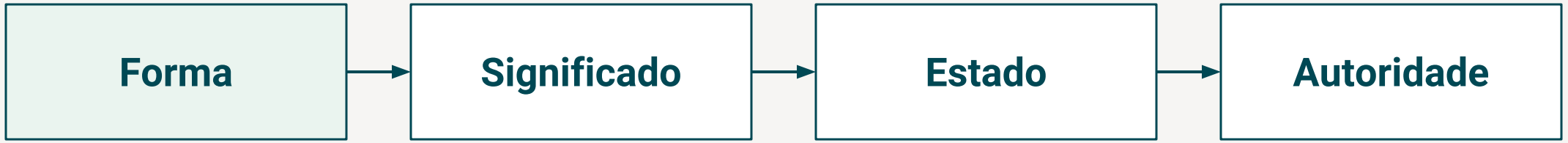
Explique por que autoridade e antecedente são verificações distintas.

# Condições para uma mudança de estado




Cada ligação exige evento compatível, antecedente confirmado e papel com permissão.

# Validação em camadas




A mensagem traz os campos esperados.

Ação válida para P001.

O antecedente já foi confirmado.

O remetente pode confirmar a ação.

As condições do consumidor permanecem visíveis. Um schema pode complementar a forma, sem substituir as regras.

# Tipos e domínio fechado: consulta

| Condição declarada | Exemplo aceito | Exemplo recusado |
| --- | --- | --- |
| Item do catálogo | camiseta | item desconhecido |
| Quantidade inteira positiva | 2 | 0 |
| Campos previstos no contrato | id e quantidade | campo inesperado |

Pydantic pode declarar essas regras. O núcleo usa dicionários e condições visíveis; tipos não concedem autoridade.

# Geração estruturada e validação local

## Durante a geração

A API pode restringir o formato ou pedir uma tool.

O recurso depende do modelo e do endpoint.

## Depois da resposta

O parser lê o resultado.

Pydantic confere o contrato declarado.

O executor verifica regra e autorização.

Pedir JSON no texto não garante geração restringida. Um objeto válido ainda pode conter uma decisão errada.

# Nome declarado e identidade verificada

## Simulação local

O dicionário declara Ana ou Bruno.

Uma tabela decide quem pode agir.

## Serviço real

A origem precisa ser autenticada.

A identidade autenticada precisa de permissão.

Trocar o texto de remetente não prova que outra pessoa ou serviço enviou a mensagem.

## Alcance da votação

# Três votos tornam uma reserva válida?

Todos os participantes concordaram, mas nenhum apresentou a confirmação do estoque.

**01** Há preferência

**02** Há confirmação

**03** Falta evidência

Compare a pergunta resolvida pela votação com a condição necessária para aplicar um efeito.

# Votação local e consenso distribuído

## Votação deste exercício

Escolhe uma opção entre as alternativas.

Roda em um processo didático.

## Consenso distribuído

Define acordo, validade e progresso sob hipóteses.

Raft replica um log. FLP delimita garantias.

O notebook não implementa Raft nem demonstra as garantias desses algoritmos. [S09, S10]

# Leitura da mensagem e do consumidor

## Qual condição o consumidor vai conferir primeiro?

**01** Leia a referência preparada de P001 e a fila.

**02** Preveja cada decisão e o estado resultante.

**03** Execute fluxo_mensagem e confira os motivos.

O núcleo é independente da execução anterior. IDs vistos e estado são reiniciados para cada comparação.

# Uma mensagem alterada por rodada

| Variação | Sua previsão | O que observar |
| --- | --- | --- |
| Inverter a ordem | Antecedente disponível? | Estado e motivo |
| Trocar remetente | Papel com autoridade? | Aceite ou recusa |
| Repetir id | Efeito já conhecido? | Quantidade de efeitos |

Preencha a previsão antes de executar. Depois explique a diferença observada.

# Dê um dono ao campo

## Duas pessoas querem confirmar a reserva

**01** Mostre uma política que aceite apenas o papel responsável.

**02** Troque só o remetente de uma mensagem e reexecute.

**03** Compare a recusa com o teste de mensagem repetida.

Uma duplicata pergunta se já processamos. Autoridade pergunta quem pode confirmar.

# Reconstrução da conversa de P001

## Uma outra dupla vai conferir seu registro

**01** Localize a solicitação e a resposta correspondente.

**02** Explique uma recusa com a condição que falhou.

**03** Declare o que sua verificação não consegue provar.

Nome de papel declarado em JSON não autentica um serviço de rede.

# O que a negociação vai receber




P001 e estado final
Mensagens aceitas ou recusadas
Motivo de cada decisão

A próxima aula parte de uma referência preparada. Estado preparado não equivale a tarefa negociada ou executada.